

API

Protocol API Specification

This document details the on-chain EVM JSON-RPC querying model and the Next.js P2P Payment API integration layer for the UnifyVault V2 protocol.

1. On-Chain EVM JSON-RPC Integration Layer

The primary protocol data and transaction execution layer is fully decentralized and client-side. The frontend interacts directly with smart contracts via JSON-RPC over HTTP/WebSocket.

1.1 Supported RPC Endpoints

- **Chain Name:** Chain ID | Public RPC Endpoint
- ****Base Mainnet**:** `8453` | `https://mainnet.base.org`
- ****Base Sepolia**:** `84532` | `https://sepolia.base.org`

1.2 On-Chain Query Reference

Client applications query protocol state through view methods on core contracts:

- ****Module Resolution**:** `ProtocolDirectory.getAddress(bytes32 name)`
- ****Asset Pricing**:** `OracleManager.getAssetPrice(address asset)`
- ****Price Freshness**:** `OracleManager.isPriceFresh(address asset)`
- ****Vault Net Asset Value**:** `PortfolioManager.calculateNAV()`
- ****Share Valuation**:** `PortfolioManager.calculateUVPrice()`
- ****Investor Cost Basis**:** `CostBasisManagerV2.costBasis(address user)`
- ****Investor Realized P&L**:** `CostBasisManagerV2.realizedPnL(address user)`
- ****Investor Unrealized P&L**:** `CostBasisManagerV2.unrealizedPnL(address user)`
- ****P2P Escrow Trade State**:** `P2PEscrowV2.getTrade(uint256 tradeId)`
- ****Orderbook State**:** `Marketplace.getOrder(uint256 orderId)`

2. Next.js P2P Payment API Routes

For off-chain peer-to-peer fiat settlement coordination (UPI / bank transfers), the application provides cryptographically protected Next.js App Router API routes located at `apps/web-v2/app/api/p2p/`.

2.1 Cryptographic Wallet Authentication

All write and retrieval API endpoints enforce wallet signature verification (`verifyWalletAuth`). Callers must sign an EIP-191 timestamped message proving ownership of `userAddress`.

2.2 Endpoint Catalog

`POST /api/p2p/payment-intent`

- ****Purpose**:** Creates or retrieves an off-chain Payment Intent with standardized UPI URIs for buyer fiat transfers.
- ****Security**:** Verifies caller is buyer or seller of the trade. Fetches authoritative trade details directly from `P2PEscrowV2` on-chain.
- ****Parameters**:** `tradeId`, `userAddress`, `signature`, `timestamp`.

`POST /api/p2p/payment-claim`

- **Purpose**: Records the buyer's off-chain payment declaration (UTR and receipt hash).
- **Security**: Restricted to `trade.buyer`. Does NOT trigger on-chain token release.
- **Parameters**: `tradeId`, `userAddress`, `signature`, `timestamp`, `utr`, `evidenceHash`.

`POST /api/p2p/payment-confirm`

- **Purpose**: Records seller confirmation of fiat receipt.
- **Security**: Restricted to `trade.seller`.
- **Parameters**: `tradeId`, `userAddress`, `signature`, `timestamp`.

`POST /api/p2p/payment-dispute`

- **Purpose**: Records dispute details and attaches evidence logs for arbitration.
- **Parameters**: `tradeId`, `userAddress`, `signature`, `timestamp`, `reason`.

`POST /api/p2p/payment-verify`

- **Purpose**: Sole verification authority in development/test environments (`ALLOW MOCK VERIFIER=true`) to process automated bank webhook attestations.
- **Security Invariant**: Strictly disabled in production unless explicit mock verifiers are enabled.

3. ERC-4337 Account Abstraction API Routes

For gasless UserOperation sponsorship and relaying, the application provides secure endpoints located at `apps/web-v2/app/api/smart-account/`.

`POST /api/smart-account/sponsor`

- **Purpose**: Evaluates UserOperation calls against the strict whitelist policy defined in `paymasterPolicy.ts`.
- **Whitelisted Operations**:
 - Batched Deposits: `[USDC.approve(Controller, amount), Controller.deposit(USDC, amount, minShares, receiver)]`
 - Batched P2P Escrow: `[UVBE.approve(P2PEscrow, amount), P2PEscrow.fundTrade(tradeId)]`
 - Single User Actions: `redeem`, `transfer`, `createTrade`, `submitPayment`, `confirmAndRelease`, `refund`, `cancelUnfundedTrade`, `raiseDispute`.
- **Security Checks**: Validates chain ID (84532), EntryPoint v0.7, zero native ETH value, and exact token approval matching. Returns paymaster signature and sponsorship metadata.

`POST /api/smart-account/bundler`

- **Purpose**: Relays signed UserOperations to the Base Sepolia ERC-4337 Bundler (`eth\_sendUserOperation`, `eth\_estimateUserOperationGas`, `eth\_getUserOperationReceipt`).
- **Security**: Sanitizes payloads and prevents unauthorized arbitrary RPC executions.

4. Flash 30s Rapid Binary Prediction API Routes

Located at `apps/web-v2/app/api/flashpulse/`:

`GET /api/flashpulse?address=0x...`

- **Purpose**: Retrieves user's off-chain Gasless Game Vault balance (`depositedUVBE`, `lockedUVBE`, `availableUVBE`).
- **Security**: Returns user balance and active locks.

`POST /api/flashpulse`

- **Purpose**: Processes gasless game vault actions:
- **DEPOSIT**: Deposits mock/real UVBE into the gasless fast-action vault.
- **WITHDRAW**: Withdraws available UVBE back to the main wallet.
- **LOCK_BET**: Locks betting collateral for an upcoming 30s round with custom multiplier parameter.

- ``SETTLE_WIN``: Credits payout calculation based on settled Oracle strike price and chosen multiplier (2x–20x).
- ``SETTLE_LOSS``: Settles round loss and clears locked collateral.
- **Parameters**: ``action``, ``address``, ``amountUVBE``, ``direction``, ``multiplier``, ``payoutUVBE``.